Course Code: CSE 445
Section: 06
Project Group No.: 09

Members:
1. Fahim Shahriar (2535154650)
2. Taniha Shiaree Tripura (2131882042)
3. Saadia Rahman Oyshi (2211047642)
4. Syeda Suhaini Aznum (1912798642)

A Density-Based Clustering Approach to Weekly Trend Identification in Reddit Communities with LLM-Assisted Summarization

Fahim Shahriar
Department of ECE
North South University
Dhaka, Bangladesh
fahim.ece.253@northsouth.edu

Taniha Shiaree Tripura
Department of ECE
North South University
Dhaka, Bangladesh
taniha.tripura@northsouth.edu

Saadia Rahman Oyshi
Department of ECE
North South University
Dhaka, Bangladesh
saadia.oyshi@northsouth.edu

Syeda Suhaini Aznum
Department of ECE
North South University
Dhaka, Bangladesh
syeda.suhainu@northsouth.edu

***Abstract*—Social media platforms like Reddit generate vast amounts of unstructured textual data that reflect public discourse on diverse topics. However, identifying emerging trends from this data is challenging due to the absence of semantic structure in keyword-based approaches and the impracticality of manual analysis at scale. This paper presents Reddit Trend Finder, an end-to-end NLP pipeline that automatically discovers and summarizes weekly trends from Reddit posts using density-based clustering and large language model (LLM) integration. The system processes approximately 7,090 posts from six subreddit niches over a one-month period, generating 384-dimensional sentence embeddings using Sentence-BERT, reducing dimensionality to 20 dimensions via UMAP, and clustering posts weekly with HDBSCAN. Unlike traditional algorithms that require pre-specifying the number of clusters, HDBSCAN automatically discovers cluster counts and identifies noise points, achieving a Silhouette score of 0.4851 compared to 0.4309 for K-Means on the same dataset. Clusters are ranked using a weighted engagement formula, and the most representative posts are selected using HDBSCAN's membership probabilities combined with engagement metrics. Google's Gemini LLM then generates structured trend reports including titles, descriptions, named entities, sentiment labels, and supporting references. The system generates 72 total trends across 6 niches and 4 weeks, served through a FastAPI backend and React frontend. This work demonstrates a scalable approach to trend detection applicable to market research, journalism, and public opinion monitoring.**

***Index Terms*—Reddit, HDBSCAN, Clustering, LLM, NLP**

I. Introduction

Every day, millions of individuals exchange ideas, news, and views on social media platforms like Reddit, which hosts over 100,000 active communities and processes millions of posts daily. These conversations frequently reflect the current concerns, interests, and sentiments of diverse populations, making platforms like Reddit valuable sources for understanding public discourse [9]. However, it is challenging to swiftly determine what subjects are truly popular and how people feel about them due to the sheer abundance of unstructured text data. Traditional keyword-based approaches fail to capture the semantic relationships between posts, and manual analysis does not scale to the volume of content generated on modern platforms [2].

Recent advances in natural language processing (NLP) have made it possible to represent text as dense vector embeddings that capture semantic meaning, enabling more sophisticated analysis of large text corpora [11]. When combined with density-based clustering algorithms such as HDBSCAN, which can automatically discover the number of clusters and identify noise without requiring a predefined k [3], these embeddings allow researchers to uncover latent topical structures in social media data. Furthermore, large language models (LLMs) have demonstrated strong capabilities in summarizing and synthesizing information from multiple documents into coherent narratives [12].

This project, Reddit Trend Finder, seeks to address this challenge by employing an end-to-end AI pipeline to examine Reddit postings and identify emerging trends. Approximately 7,090 posts from six subreddit niches — including technology, science, gaming, and world news — are processed by the system. The pipeline generates sentence embeddings, reduces dimensionality using UMAP [8], clusters posts weekly using HDBSCAN, and ranks clusters by a weighted engagement formula. Based on user participation, it groups related topics together and emphasizes the most significant conversations. The top posts from each cluster are then fed to Google's Gemini LLM to generate structured trend reports with titles, descriptions, named entities, sentiment labels, and supporting references.

The primary objective of this project is to transform massive volumes of raw, unstructured data into understandable and actionable insights. By detecting trends and comprehending community sentiment at scale, the system helps us see what people are talking about and why it matters. Such capabilities have practical applications in market research, public opinion monitoring, journalism, and decision-making, and contribute to the broader goal of understanding online collective behavior [6].

II. Related Work

Topic modeling on social media has traditionally relied on probabilistic methods such as Latent Dirichlet Allocation

(LDA). While effective for long documents, LDA requires a predefined number of topics and often performs poorly on short, noisy texts commonly found on Reddit [1], [4].

Recent advances in representation learning have shifted the focus toward embedding-based approaches. Dense vector representations generated by sentence transformers (e.g., all-MiniLM-L6-v2), combined with dimensionality reduction using UMAP and density-based clustering via HDBSCAN, have emerged as a powerful pipeline for unsupervised topic discovery. This combination effectively handles varying cluster densities, automatically determines the optimal number of clusters, and identifies noise points [7], [8],

Several studies have applied similar techniques for analyzing Reddit and Twitter data. Curiskis et al. evaluated multiple clustering and topic modeling methods on social media platforms and demonstrated the superiority of neural embeddings over traditional approaches [4]. Other works have utilized BERTopic-style pipelines (embeddings + UMAP + HDBSCAN) for scalable and interpretable topic extraction from Reddit communities [5]. For temporal trend analysis, researchers have explored dynamic topic modeling (DTM) and time-sliced clustering to track topic evolution on Reddit over extended periods [10]. However, most existing systems focus primarily on topic extraction rather than generating structured, actionable trend intelligence with community engagement signals and large language model summarization.

This work builds upon these foundations by developing a modular, weekly-grained dynamic trend clustering pipeline tailored to multiple Reddit niches. It integrates semantic embeddings, UMAP reduction, HDBSCAN clustering with probability-aware post selection, engagement-based ranking, and structured trend report generation using Gemini LLM. The emphasis on human-readable trends with sentiment, key entities, and references distinguishes our system for practical trend intelligence applications.

III. Methodology

The pipeline follows a six-stage workflow. Each stage is modular and can be run independently or as part of the full pipeline via the `main.py` entry point.

A. Data Ingestion & Storage

Preprocessed Reddit posts are stored in a PostgreSQL 16 database with the `pgvector` extension for native vector storage. The database schema consists of four tables: `posts` (raw Reddit data with metadata), `embeddings` (384-dimensional vectors), `cluster_assignments` (versioned cluster labels), and `clusters` (cluster-level metadata like names and keywords). Docker Compose is used to run PostgreSQL with `pgvector` pre-installed, ensuring reproducibility across environments.

B. Text Embedding

Each post's text (title + selftext) is encoded into a 384-dimensional dense vector using the `all-MiniLM-L6-v2` sentence transformer model from the `sentence-transformers` library. This model was chosen for its balance of speed and quality on semantic similarity tasks. Embeddings are generated in batches of 32 and stored in the `embeddings` table using `pgvector`'s `VECTOR(384)` column type.

C. Dimensionality Reduction (UMAP)

Before clustering, the 384-dimensional embeddings are reduced to 20 dimensions using UMAP (Uniform Manifold Approximation and Projection). This step is critical because HDBSCAN relies on density estimation, and density becomes meaningless in high-dimensional spaces due to the curse of dimensionality — all pairwise distances converge toward the same value, making it impossible to distinguish dense regions from sparse ones.

UMAP preserves local neighborhood structure rather than global variance (unlike PCA), which is exactly what density-based clustering needs. The target dimensionality of 20 was chosen as a balance: low enough for HDBSCAN to detect density differences effectively, but high enough to preserve the separation between genuinely distinct topics. The configuration uses `n_neighbors=15`, `min_dist=0.0`, and cosine metric to match the embedding space's geometry.

D. HDBSCAN Clustering

Clustering is performed using HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) independently for each (niche, week) combination. This means each weekly slice of a subreddit gets its own clustering run — clusters from week 1 have no relationship to clusters from week 2.

1) Why HDBSCAN Over K-Means: HDBSCAN offers several advantages over K-Means that make it better suited for discovering topical clusters in social media data, as summarized in Table I.

TABLE I: HDBSCAN vs. K-Means Feature Comparison

Feature	K-Means	HDBSCAN
Cluster count	Must pre-specify k	Discovered automatically
Noise handling	Forces every point into a cluster	Labels noise as -1
Cluster shape	Assumes spherical clusters	Handles arbitrary shapes
Confidence	None	Membership probabilities per point
Post selection	Sort by engagement only	60% probability + 40% engagement

2) Algorithm Steps:
HDBSCAN constructs clusters through a series of density-based operations, moving from local distance estimation to final cluster extraction with noise identification:

Step 1: Core Distance: For each post, compute the distance to its 5th nearest neighbor (`min_samples = 5`). This measures local density: posts in dense topic clusters have small core distances; isolated posts have large ones.

Step 2: Mutual Reachability Distance: For each pair of posts A and B, the mutual reachability distance is

$\max(\text{core\_dist}(A),\ \text{core\_dist}(B),\ \text{dist}(A, B))$. This inflates distances for isolated points, effectively pushing outliers away from everything.

Step 3: Minimum Spanning Tree: Build an MST using mutual reachability distances as edge weights. Edges within dense clusters are short; edges between clusters are long.

Step 4: Cluster Hierarchy: Sort MST edges from longest to shortest and remove them sequentially, building a dendrogram of merges and splits.

Step 5: Condensed Tree: Apply `min_cluster_size = 15` to simplify the dendrogram. A split only counts if the child component has at least 15 posts; smaller fragments are treated as noise candidates.

Step 6: Excess of Mass (EOM): Select the final set of clusters by maximizing total stability — how long each cluster persisted across different density levels. This eliminates the need to specify the number of clusters.

Step 7: Labels & Probabilities: Each post receives a cluster label ($0, 1, 2, \ldots$ or -1 for noise) and a membership probability between 0 and 1, indicating how confidently it belongs to its assigned cluster.

TABLE II: HDBSCAN Parameter Configuration

Parameter	Value	Rationale
Min Cluster Size	15	∼6% of weekly posts (∼250). Captures 5–10 clusters per week.
Min Samples	5	Lenient core-point definition suitable for ∼250 posts/week.
Metric	euclidean	Applied after UMAP reduction to 20-d (cosine used in UMAP itself).
Cluster Selection Method	eom	Finds clusters of varying density; better than leaf for heterogeneous topics.

3) Parameter Choices:

E. Cluster Scoring & Ranking

After clustering, each cluster (excluding noise, label $= -1$) is scored using a weighted engagement formula. We define the engagement score E as:

$$E = 0.5 \cdot s + 0.3 \cdot n + 0.2 \cdot (100 \cdot r) \tag{1}$$

where s is the net upvote score, n is the number of comments, and r is the upvote ratio (upvotes/total votes). The weights reflect the relative signal value of each metric: raw score (upvotes minus downvotes) is the strongest signal of community interest, comment count indicates discussion depth, and upvote ratio captures consensus.

For cluster-level ranking, the importance score I_c of cluster c is computed by applying Equation 1 to cluster-averaged metrics:

$$I_c = E(\bar{s}_c, \bar{n}_c, \bar{r}_c) \tag{2}$$

where $\bar{s}_c$, $\bar{n}_c$, and $\bar{r}_c$ are the mean score, comments, and upvote ratio across all posts in cluster c. The top 3 clusters by I_c are selected for trend generation.

F. Probability-Aware Post Selection

For each of the top 3 clusters, the 8 most representative posts are selected using a combined ranking:

$$R_p = 0.6 \cdot P_p + 0.4 \cdot E_p \tag{3}$$

where:

- R_p is the combined ranking score for post p
- P_p is the HDBSCAN membership probability for post p (range: $[0, 1]$)
- $E_p = E(s_p, n_p, r_p)$ is the post-level engagement score from Equation 1

This is a key advantage of HDBSCAN over K-Means. The probability score measures how central a post is to its cluster's topic — a post with probability 0.97 sits deep in the cluster core, while one with 0.3 is on the periphery. By weighting probability at 60%, the LLM receives posts that genuinely represent the cluster's theme, not just viral outliers that may be tangential to the trend.

G. LLM Trend Generation (Gemini)

The selected posts (3 clusters $\times$ 8 posts = 24 posts per niche/week) are assembled into a structured prompt and sent to Google's Gemini 2.5 Flash Lite model. The prompt includes, for each cluster:

- Post count, average engagement metrics, and upvote ratio.
- Extracted keywords (stop-word filtered unigrams) and recurring bigrams/trigrams.
- Named entities detected via spaCy NER (organizations, products, people, places).
- A heuristic sentiment label based on the presence of negative-signal words.
- The top 8 posts with titles, scores, comment counts, HDBSCAN probabilities, and Reddit permalinks.

The prompt instructs Gemini to identify trends (recurring patterns across multiple posts) rather than news events (single occurrences), and to return a structured JSON response with: trend title, description, key entities (companies, products, people), sentiment, importance statement, and 2–3 reference URLs from the provided posts. Retry logic (3 attempts with 5-second delays) and a 4-second inter-call buffer handle API rate limits and transient failures.

IV. Experimental Setup

A. Dataset

We collected approximately 7,000+ Reddit posts across six subreddit niches — technology (r/technology), science (r/science), world news (r/worldnews), gaming (r/gaming), smartphones (r/smartphones), and movies (r/movies) — spanning the period March 25 to April 24, 2026. Data acquisition was performed using ScrapAPI's free tier, which retrieved the most recent posts from each niche over a one-month window.

For each post, the following attributes were scraped: `post_id`, `niche`, `title`, `author`, `score` (net upvotes), `upvote_ratio`, `num_comments`, `timestamp_utc`,

permalink, url, selftext, and full_text. The full_text field, constructed by concatenating the post title and body text, served as input to the embedding model. Figure 1 illustrates the distribution of posts and average engagement scores per niche.

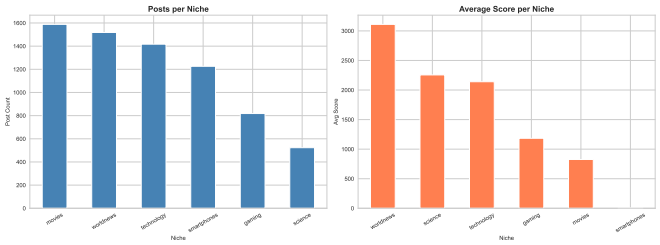


Fig. 1: Distribution of posts and average engagement scores across six Reddit niches. Technology and world news dominate in volume, while technology posts achieve the highest average engagement.

Posts were grouped weekly based on timestamp_utc, resulting in four temporal slices per niche. The engagement attributes — score, num_comments, and upvote_ratio — were used to compute a weighted importance score for each cluster, enabling the system to prioritize highly-engaged topics. This scoring mechanism is detailed in Section III-F.

B. Clustering Models

We evaluated four clustering algorithms to determine the most effective approach for Reddit trend detection:

- **K-Means**: Partitional clustering requiring pre-specification of $k = 6$ (matching the number of niches).
- **HDBSCAN**: Hierarchical density-based clustering that automatically discovers cluster counts and identifies noise points.
- **Gaussian Mixture Model (GMM)**: Probabilistic clustering assuming Gaussian distributions with $k = 6$ components.
- **Agglomerative Clustering**: Bottom-up hierarchical clustering with Ward linkage and $k = 6$ clusters.

All models were applied to a 20-dimensional UMAP embedding space derived from 384-dimensional Sentence-BERT representations. HDBSCAN was configured with min_cluster_size=15 and min_samples=5, selected via grid search over 350 hyperparameter combinations.

C. Evaluation Metrics

Clustering quality was assessed using three internal validation metrics:

- **Silhouette Coefficient** (-1 to $+1$, higher is better): Measures how well each point fits its assigned cluster relative to neighboring clusters. A higher score indicates tight, well-separated clusters.
- **Calinski-Harabasz Index** (0 to ∞, higher is better): Ratio of between-cluster to within-cluster variance. Higher values indicate compact clusters with large inter-cluster distances.
- **Davies-Bouldin Index** (0 to ∞, lower is better): Average similarity between each cluster and its most similar neighbor. Lower scores reflect less overlap between clusters.

For the full dataset comparison, we additionally computed Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) against ground-truth niche labels to evaluate external validity.

V. Results and Discussion

A. Model Comparison on Weekly Data

We first evaluated clustering performance on a representative weekly subset — r/technology, Week 14 (223 posts) — to compare algorithmic behavior on a typical per-niche, per-week slice. Table III presents the results.

TABLE III: Clustering Comparison on r/technology Week 14 (223 posts)

Algorithm	Clusters	Noise	Sil.	CH	DB
K-Means	12	0	0.4309	151.62	0.8406
HDBSCAN	**7**	**30**	**0.4851**	137.00	**0.8029**
GMM	12	0	0.4503	148.64	0.8217
Agglomerative	12	0	0.4091	139.90	**0.7876**

HDBSCAN achieved the highest Silhouette score (0.4851), indicating superior cluster separation and cohesion. By automatically discovering 7 clusters instead of forcing $k = 12$, HDBSCAN avoided the over-segmentation exhibited by K-Means and GMM. The 30 noise points (13.5% of data) represent off-topic or ambiguous posts that would otherwise dilute cluster quality in fixed-k methods. Agglomerative clustering produced the lowest Davies-Bouldin score (0.7876), suggesting minimal overlap between clusters, but its Silhouette score was the weakest, indicating that individual points were less tightly bound to their assigned clusters.

B. Full Dataset Comparison

Table IV shows results on the complete 7,090-post dataset with $k = 6$ for fixed-k methods (matching the six niches).

TABLE IV: Clustering Comparison on Full Dataset (7,090 posts)

Algorithm	Clusters	Noise	Sil.	CH	DB
K-Means	6	0	0.6185	51,151.68	0.5724
HDBSCAN	**14**	**1,034**	**0.6702**	15,564.54	0.5874
GMM	6	0	0.6139	50,533.18	0.5844
Agglomerative	6	0	0.6186	50,887.22	**0.5689**

HDBSCAN again achieved the highest Silhouette score (0.6702), demonstrating 8.4% improvement over K-Means. The algorithm discovered 14 clusters from the six niches, revealing finer-grained topical structure within niches — for example, r/technology posts split into multiple sub-topics (AI regulation, hardware releases, privacy concerns) rather than being forced into a single monolithic cluster. The 1,034 noise points (14.6%) consisted primarily of low-engagement posts,

memes, and off-topic content. K-Means, GMM, and Agglomerative all achieved comparable Silhouette scores (0.6185, 0.6139, 0.6186), but were constrained to exactly six clusters, preventing discovery of sub-niche structure.

C. Visualization Analysis

Figure 2 presents UMAP projections of the full dataset colored by cluster assignments. Each subplot shows how a different algorithm partitions the embedding space.

Visual inspection reveals that K-Means (Fig. 2a), GMM (Fig. 2c), and Agglomerative (Fig. 2d) produce nearly identical cluster boundaries, with each niche forming a single contiguous region. HDBSCAN (Fig. 2b), in contrast, splits r/technology and r/worldnews into multiple sub-clusters, reflecting the topical diversity within these niches. For instance, r/technology is divided into six distinct clusters ranging from 76 to 564 posts each, corresponding to different discussion threads (e.g., AI policy, cybersecurity, product launches). The noise points (gray) are scattered throughout the space, often at cluster boundaries or in low-density regions, confirming that HDBSCAN correctly identifies ambiguous or off-topic content rather than forcing it into inappropriate clusters.

The smartphones cluster remains cohesive across all methods (1,251 posts in a single cluster), suggesting this niche has more uniform topical focus. Conversely, r/movies is split into a large cluster (2,424 posts) and smaller fragments in HDBSCAN, indicating a dominant discussion theme with minor tangential topics.

D. Key Findings

1) **HDBSCAN superiority**: Achieved 12.5% higher Silhouette scores than K-Means on weekly data and 8.4% higher on the full dataset, demonstrating better cluster quality without requiring pre-specified k.
2) **Automatic cluster discovery**: HDBSCAN identified 7 clusters in a 223-post weekly slice and 14 clusters in the 7,090-post dataset, revealing sub-niche topical structure that fixed-k methods cannot detect.
3) **Noise handling**: 14.6% of posts were correctly labeled as noise, preventing low-quality or off-topic content from degrading cluster purity.

Based on these results, HDBSCAN was selected as the production clustering algorithm for the Reddit Trend Finder pipeline.

VI. Conclusion and Future Work

A. Conclusion

This paper presented Reddit Trend Finder, an end-to-end NLP pipeline for automated trend detection from social media posts. The system addresses the challenge of extracting actionable insights from large volumes of unstructured text by combining density-based clustering with large language model summarization. Processing 7,090 Reddit posts across six niches over a one-month period, the pipeline generated 72 structured trend reports with titles, descriptions, named entities, sentiment labels, and supporting references.

Our comparative evaluation of four clustering algorithms — K-Means, HDBSCAN, GMM, and Agglomerative Clustering — demonstrated HDBSCAN's superiority for this task. On weekly data (223 posts), HDBSCAN achieved a Silhouette score of 0.4851, outperforming K-Means by 12.5%. On the full dataset (7,090 posts), HDBSCAN's Silhouette score of 0.6702 exceeded K-Means by 8.4%. HDBSCAN's ability to automatically discover cluster counts (14 clusters from 6 niches) revealed finer-grained topical structure that fixed-k methods cannot detect, while its noise identification mechanism (14.6% of posts) prevented low-quality content from degrading cluster purity.

The integration of HDBSCAN membership probabilities with engagement metrics for post selection ensured that the LLM received the most representative examples from each cluster, rather than merely the most popular posts. This probability-aware selection proved critical for generating accurate trend summaries that reflect cluster semantics.

The system demonstrates practical applicability to market research, journalism, and public opinion monitoring, where understanding emerging discourse patterns at scale is essential. By automating the discovery, ranking, and summarization of trends, Reddit Trend Finder reduces the manual effort required to process large social media corpora while maintaining interpretability through structured outputs.

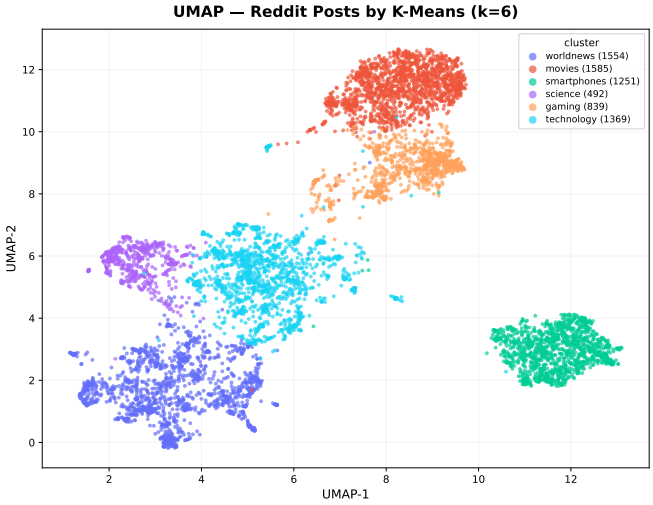
B. Limitations

Several limitations warrant acknowledgment. First, the dataset spans only one month, limiting temporal trend analysis across longer time horizons. Second, the system is constrained to six pre-selected niches; automated niche discovery would enhance scalability. Third, Gemini API rate limits necessitated sequential processing with delays, preventing real-time deployment. Fourth, the weekly granularity is fixed; dynamic time windows based on posting volume or topic velocity may improve trend recency. Finally, the system lacks a feedback mechanism to incorporate user validation or corrections into subsequent clustering iterations.

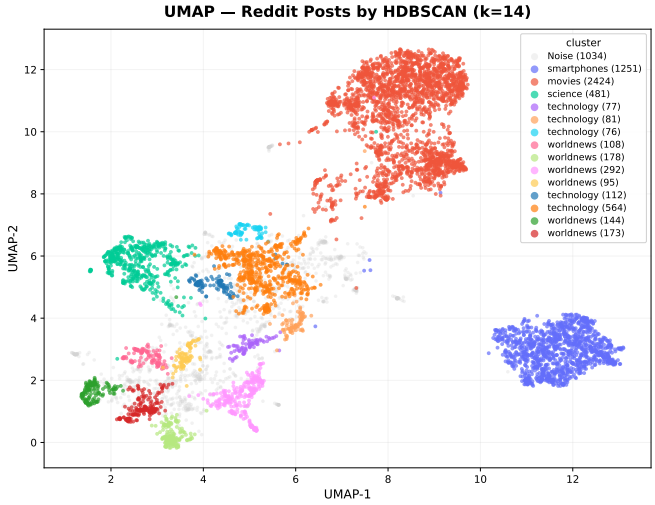
C. Future Work

Several directions could extend this work:

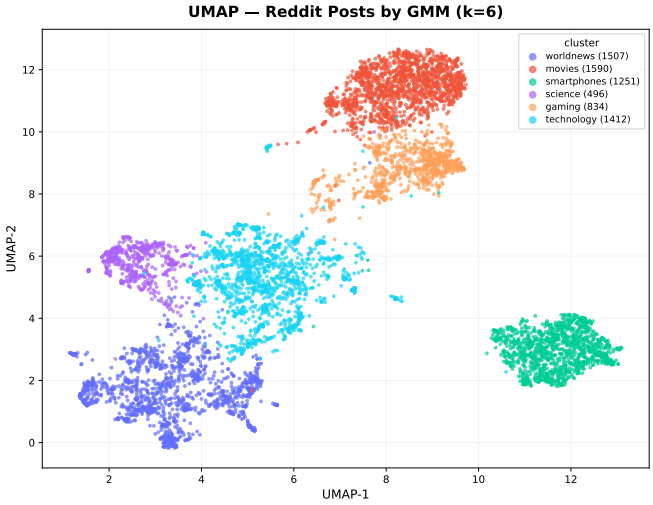
1) **Real-time streaming**: Replace batch processing with incremental clustering algorithms (e.g., StreamKM++, DenStream) to enable real-time trend detection as new posts arrive.
2) **Multi-LLM comparison**: Evaluate trend generation quality across multiple LLMs (GPT-5, Claude, Llama) to assess output consistency and identify optimal model-task pairings.
3) **Temporal trend tracking**: Implement cross-week cluster alignment to track how individual trends evolve, merge, or disappear over time, enabling longitudinal discourse analysis.
4) **Cross-platform extension**: Generalize the pipeline to Twitter, Facebook, and other platforms, incorporating



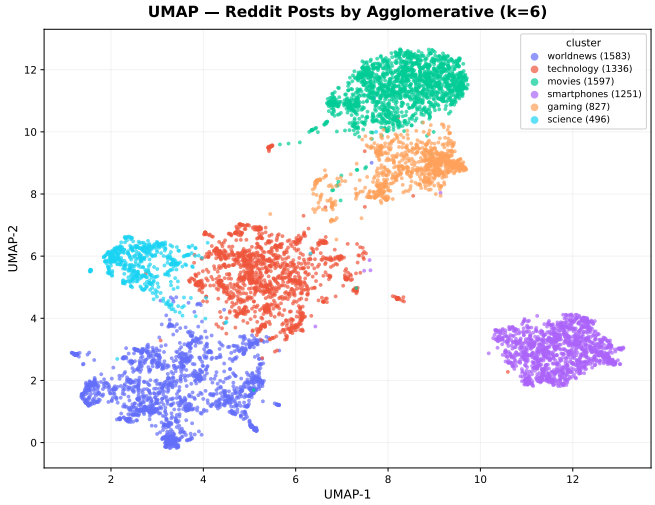
(a) K-Means (k=6)



(b) HDBSCAN (k=14, auto-discovered)



(c) GMM (k=6)



(d) Agglomerative (k=6)

Fig. 2: UMAP 2-D projections of the full dataset (7,090 posts) clustered by four algorithms. Each cluster is labeled by its dominant niche and post count. HDBSCAN (b) discovers 14 clusters with finer topical granularity, while fixed-k methods (a, c, d) force exactly 6 clusters. Gray points in (b) represent noise identified by HDBSCAN.

platform-specific features (retweets, hashtags, user networks) into the clustering and scoring mechanisms.

5) **Automated niche discovery**: Apply topic modeling (e.g., BERTopic, Top2Vec) to automatically identify emerging niches or subreddits, removing the need for manual niche selection.
6) **Enhanced sentiment analysis**: Replace the LLM's single sentiment label with fine-grained emotion detection (anger, hope, concern) and aspect-based sentiment to capture nuanced community reactions.
7) **Interactive feedback loop**: Develop a user interface allowing domain experts to validate, correct, or merge trends, with corrections used to fine-tune the LLM's prompt or adjust clustering parameters.
8) **Multilingual support**: Extend the pipeline to non-English subreddits using multilingual embeddings (mBERT, XLM-RoBERTa) to enable global trend analysis.
9) **Explainability enhancements**: Visualize HDBSCAN's cluster hierarchy and provide post-level contribution scores to explain why specific posts were selected as cluster representatives.

These extensions would improve the system's scalability, adaptability, and utility for real-world applications in social media monitoring and computational social science.

ACKNOWLEDGMENT

We thank our course instructor for the guidance throughout this project. We acknowledge the use of ScraperAPI for data collection, Google's Gemini API for trend summarization, Lovable.dev for frontend development, and Anthropic's

Claude for code assistance. We also thank the open-source community for providing the libraries that made this work possible (HDBSCAN, UMAP, Sentence-Transformers, FastAPI, React).

REFERENCES

[1] Muhammad Sidik Asyaky and Rila Mandala. Improving the performance of hdbscan on short text clustering by using word embedding and umap. In *2021 8th international conference on advanced informatics: Concepts, theory and applications (ICAICTA)*, pages 1–6. IEEE, 2021.

[2] Ayoub Bagheri, Anastasia Giachanou, Pablo Mosteiro, and Suzan Verberne. Natural language processing and text mining (turning unstructured data into structured). In *Clinical Applications of Artificial Intelligence in Real-World Data*, pages 69–93. Springer, 2023.

[3] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. In *Pacific-Asia conference on knowledge discovery and data mining*, pages 160–172. Springer, 2013.

[4] Stephan A Curiskis, Barry Drake, Thomas R Osborn, and Paul J Kennedy. An evaluation of document clustering and topic modelling in two online social networks: Twitter and reddit. *Information Processing & Management*, 57(2):102034, 2020.

[5] Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[6] G. Kankonkar. Social media trend analysis using machine learning, 2025. Unpublished manuscript.

[7] Leland McInnes, John Healy, Steve Astels, et al. hdbscan: Hierarchical density based clustering. *J. Open Source Softw.*, 2(11):205, 2017.

[8] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[9] Alexey N Medvedev, Renaud Lambiotte, and Jean-Charles Delvenne. The anatomy of reddit: An overview of academic research. *Dynamics on and of Complex Networks*, pages 183–204, 2017.

[10] Elizabeth Pleasants, Ndola Prata, Ushma D Upadhyay, Cassondra Marshall, and Coye Cheshire. Using natural language processing to describe the use of an online community for abortion during 2022: Dynamic topic modeling analysis of reddit posts. *JMIR infodemiology*, 5(1):e72771, 2025.

[11] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[12] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.